

Rapport de Projet : Modélisation Géométrique

Simulation d'une Invasion Zombie dans l'Espace

BACCON Noah - BRAHIM JEDOU Cheikh / Groupe B1

18 mai 2026

Table des matières

1	Introduction	2
2	Modélisation de l'Environnement et des Particules	2
2.1	Initialisation et Mouvement Aléatoire	2
2.2	Calcul des Distances et Voisinage (KDTree)	2
3	Propagation de l'Infection (Effet Trafalgar)	2
4	Règles Comportementales et Forces	3
4.1	Le Modèle d'Aoki	3
4.2	Dynamiques d'Invasion : Chasse et Fuite	3
4.3	Cône de Vision	3
5	Implémentation et Visualisation	3
6	Conclusion	4

1 Introduction

Ce projet a pour but de représenter un système de particules en 3D. Inspirée des modèles de déplacement des bancs de poissons, notre simulation est orientée vers la science-fiction : une invasion zombie à bord d'un vaisseau spatial.

La simulation met en scène des "explorateurs" (particules saines) et des "zombies" (particules infectées) bougeant dans un espace 3D confiné, en apesanteur. Les individus interagissent selon des règles locales de répulsion, d'alignement et d'attraction, influencées par leur état (sain ou infecté).

2 Modélisation de l'Environnement et des Particules

2.1 Initialisation et Mouvement Aléatoire

Chaque individu est modélisé par une particule possédant un identifiant, une position vectorielle $\vec{p}_i = (x_i, y_i, z_i)$, une vitesse $\vec{v}_i = (v_{xi}, v_{yi}, v_{zi})$, un état (sain ou zombie) et une caractéristique de rapidité (explorateur standard ou rapide). Le mouvement est simulé en mettant à jour la position à chaque intervalle de temps Δt :

$$\vec{p}_i(t + \Delta t) = \vec{p}_i(t) + \vec{v}_i(t) \times \Delta t$$

Les particules évoluent dans une zone cubique délimitée. Lorsqu'une particule atteint les frontières de cette zone, sa vitesse sur l'axe correspondant est inversée pour simuler un rebond sur les parois du vaisseau.

2.2 Calcul des Distances et Voisinage (KDTree)

Il est nécessaire d'identifier les voisins proches de chaque expéditeur. La distance euclidienne entre deux particules i et j est définie par :

$$d(p_i, p_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + (z_i - z_j)^2}$$

Afin d'éviter une complexité quadratique $O(N^2)$ lors de la recherche des voisins, nous avons implémenté un arbre de recherche spatial (KDTree) via la bibliothèque `scipy.spatial`.

3 Propagation de l'Infection (Effet Trafalgar)

La transition de l'état "sain" à "zombie" modélise une forme de contagion par proximité, inspirée de la propagation des mouvements (effet Trafalgar) dans les bancs de poissons.

Initialement, un individu est désigné aléatoirement comme le "patient zéro". À chaque itération, si un explorateur sain se trouve à une distance inférieure au rayon d'infection (R_{infect}) d'un zombie, il est immédiatement contaminé et change d'état. Ce changement d'état modifie son comportement et sa vitesse maximale.

4 Règles Comportementales et Forces

4.1 Le Modèle d'Aoki

Lorsque les particules sont du même état (entre explorateurs ou entre zombies), elles obéissent aux trois règles comportementales de base proposées par Ichiro Aoki :

- **Évitement (Répulsion)** : Pour éviter les collisions, si la distance est inférieure au rayon de répulsion (R_{rep}), une force éloigne la particule i de j :

$$\vec{F}_{rep} = -k_{rep} \cdot \frac{\vec{p}_j - \vec{p}_i}{\|\vec{p}_j - \vec{p}_i\|}$$

- **Alignement** : Si la distance est intermédiaire ($R_{rep} \leq d < R_{align}$), la particule aligne sa vitesse sur celle de ses voisins pour créer un mouvement de groupe :

$$\vec{F}_{align} = \frac{k_{align}}{N_{align}} \sum_j \vec{v}_j$$

- **Attraction** : Si la distance est grande mais dans le champ de perception ($R_{align} \leq d < R_{att}$), la particule est attirée par ses voisins pour maintenir la cohésion :

$$\vec{F}_{att} = k_{att} \cdot \frac{\vec{p}_j - \vec{p}_i}{\|\vec{p}_j - \vec{p}_i\|}$$

4.2 Dynamiques d'Invasion : Chasse et Fuite

Nous avons ensuite introduit de nouvelles interactions pour se rapprocher d'une invasion zombie :

- **Fuite** : Les explorateurs sains ressentent une forte force de répulsion (fuite) face aux zombies qui entrent dans leur rayon de perception.
- **Chasse** : À l'inverse, les zombies sont attirés par les explorateurs non contaminés à portée et se dirigent agressivement vers elles.

4.3 Cône de Vision

Pour rendre plus réaliste, un cône de vision peut être ajouté. Au lieu de percevoir à 360°, les particules ne sont influencées que par les entités situées dans leur cône de vision. Le cône est déterminé par le produit scalaire entre le vecteur direction de la particule et le vecteur la reliant à sa cible :

$$\cos(\theta) = \frac{\vec{v}_i}{\|\vec{v}_i\|} \cdot \frac{\vec{p}_j - \vec{p}_i}{\|\vec{p}_j - \vec{p}_i\|}$$

Seules les particules pour lesquelles ce produit scalaire dépasse un certain seuil (par exemple, un angle inférieur à 60°) sont prises en compte.

5 Implémentation et Visualisation

Le projet a été développé en Python. L'optimisation spatiale est gérée par `scipy.spatial.KDTree`.

La visualisation 3D de l'espace confiné (le vaisseau) est réalisée à l'aide de `plotly.graph_objects`. Chaque image de l'animation met à jour la position et la couleur/forme des particules : les

explorateurs sains sont en bleu (ou or pour les rapides), tandis que les infectés prennent instantanément la couleur rouge et une forme de croix. Ce choix a été fait après un début sur Matplotlib car Plotly nous semblait plus fourni et facile d'utilisation pour l'animation.

6 Conclusion

Ce projet a permis de mettre en application avec succès les modèles mathématiques liés aux particules et de les étendre à un sujet qui nous plaisait.